

ARRAYS

Aim:

To understand and implement array operations in Java.

PRE LAB EXERCISE

QUESTIONS

1. What is an array?

An **array** is a data structure that stores **multiple values of the same data type** under a **single name**, and each value is accessed using an **index**.

Example:

```
int marks[5] = {80, 75, 90, 85, 70};
```

2. Why are arrays used?

Arrays are used because they:

- Store **many related values** using one variable name
- Make programs **shorter and easier to manage**
- Allow **easy access** to data using index positions
- Are useful for **loops, searching, sorting, and calculations**
- Improve **memory organization**

3. What is the difference between array and variable?

Feature	Variable	Array
Stores	Single value	Multiple values

Feature	Variable	Array
Data type	One value of a type	Multiple values of same type
Memory	One memory location	Continuous memory locations
Access	By variable name	By index (e.g., a[0])
Example	int x = 10;	int a[3] = {1,2,3};

IN LAB EXERCISE

Objective:

To perform array operations using simple programs.

PROGRAMS:

1. Program to Read and Print Array Elements

Code:

```
import java.util.Scanner;

public class ReadPrintArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[5];
        System.out.println("Enter 5 elements:");
        for(int i = 0; i < 5; i++)
            arr[i] = sc.nextInt();
        System.out.println("Array elements are:");
        for(int i = 0; i < 5; i++)
            System.out.print(arr[i] + " ");
    }
}
```

OUTPUT:

<terminated> Class1 [Java Application] C:\Users\SWETHA\.p2\pool\plugins\org.eclipse.j

Enter 5 elements:

1 2 3 4 5

Array elements are:

1 2 3 4 5

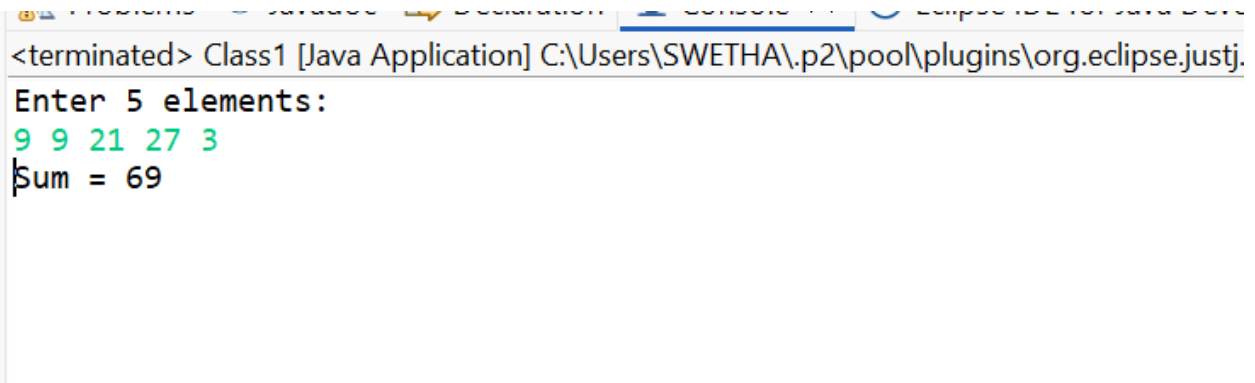
2. Program to Find Sum of Array Elements

Code:

```
import java.util.Scanner;

public class SumArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[5];
        int sum = 0;
        System.out.println("Enter 5 elements:");
        for(int i = 0; i < 5; i++)
            arr[i] = sc.nextInt();
        for(int i = 0; i < 5; i++)
            sum += arr[i];
        System.out.println("Sum = " + sum);
    }
}
```

OUTPUT:



```
<terminated> Class1 [Java Application] C:\Users\SWETHA\.p2\pool\plugins\org.eclipse.justj.  
Enter 5 elements:  
9 9 21 27 3  
Sum = 69
```

3. Program to Find Largest Element in an Array

Code:

```
import java.util.Scanner;  
  
public class LargestElement {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int[] arr = new int[5];  
        System.out.println("Enter 5 elements:");  
        for(int i = 0; i < 5; i++)  
            arr[i] = sc.nextInt();  
        int max = arr[0];  
        for(int i = 1; i < 5; i++)  
            if(arr[i] > max)  
                max = arr[i];  
        System.out.println("Largest element = " + max);  
    }  
}
```

OUTPUT:

<terminated> Class1 [Java Application] C:\Users\SWEIHA\p2\pool\plugins\org.eclipse.justj.op

Enter 5 elements:

9 9 21 27 3

Largest element = 27

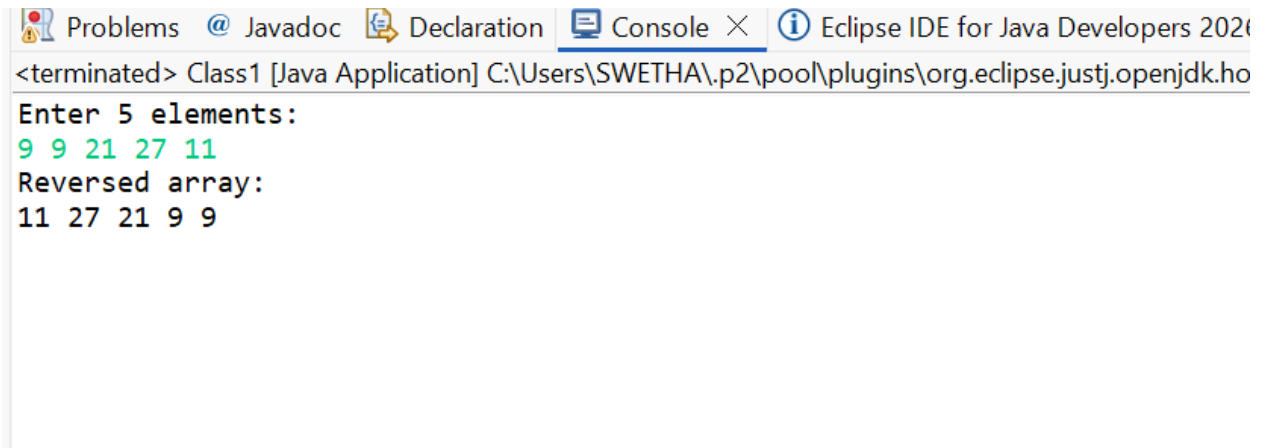
4. Program to Reverse an Array

Code:

```
import java.util.Scanner;

public class ReverseArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[5];
        System.out.println("Enter 5 elements:");
        for(int i = 0; i < 5; i++)
            arr[i] = sc.nextInt();
        System.out.println("Reversed array:");
        for(int i = 4; i >= 0; i--)
            System.out.print(arr[i] + " ");
    }
}
```

OUTPUT:



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The console output displays the execution of a Java program. It starts with a prompt 'Enter 5 elements:' followed by the input '9 9 21 27 11'. Then, it shows 'Reversed array:' followed by the output '11 27 21 9 9'. The IDE title bar indicates 'Eclipse IDE for Java Developers 2021'.

```
<terminated> Class1 [Java Application] C:\Users\SWETHA\.p2\pool\plugins\org.eclipse.justj.openjdk.ho
Enter 5 elements:
9 9 21 27 11
Reversed array:
11 27 21 9 9
```

5. Program to Count Even and Odd Numbers

Code:

```
import java.util.Scanner;

public class EvenOddCount {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int[] arr = new int[5];

        int even = 0, odd = 0;

        System.out.println("Enter 5 elements:");

        for(int i = 0; i < 5; i++)

            arr[i] = sc.nextInt();

        for(int i = 0; i < 5; i++) {

            if(arr[i] % 2 == 0)

                even++;

            else

                odd++;

        }

        System.out.println("Even = " + even);

        System.out.println("Odd = " + odd);

    }

}
```

```
}
```

OUTPUT:

```
<terminated> Class1 [Java Application] C:\Users\SWETHA\.p2\pool\plugins\org.eclipse.jus
Enter 5 elements:
9 9 21 27 13
Even = 0
Odd = 5
```

6. Program to Sort Array in Ascending Order

Code:

```
import java.util.Scanner;

public class SortArray {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int[] arr = new int[5];

        int temp;

        System.out.println("Enter 5 elements:");

        for(int i = 0; i < 5; i++)

            arr[i] = sc.nextInt();

        for(int i = 0; i < 5; i++) {

            for(int j = i + 1; j < 5; j++) {

                if(arr[i] > arr[j]) {

                    temp = arr[i];

                    arr[i] = arr[j];

                    arr[j] = temp;

                }

            }

        }

    }

}
```

```

        }
    }
    System.out.println("Sorted array:");
    for(int i = 0; i < 5; i++)
        System.out.print(arr[i] + " ");
    }
}

```

OUTPUT:

```

<terminated> Class1 [Java Application] C:\Users\SWETHA\.p2\pool\plugins\org.eclipse.justj.op
Enter 5 elements:
9 9 27 21 10
Sorted array:
9 9 10 21 27

```

7. Program to Find Second Largest Element

Code:

```

import java.util.Scanner;

public class SecondLargest {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int[] arr = new int[5];

        System.out.println("Enter 5 elements:");

        for (int i = 0; i < 5; i++) {

            arr[i] = sc.nextInt();

        }

        int largest = Integer.MIN_VALUE;

        int second = Integer.MIN_VALUE;
    }
}

```

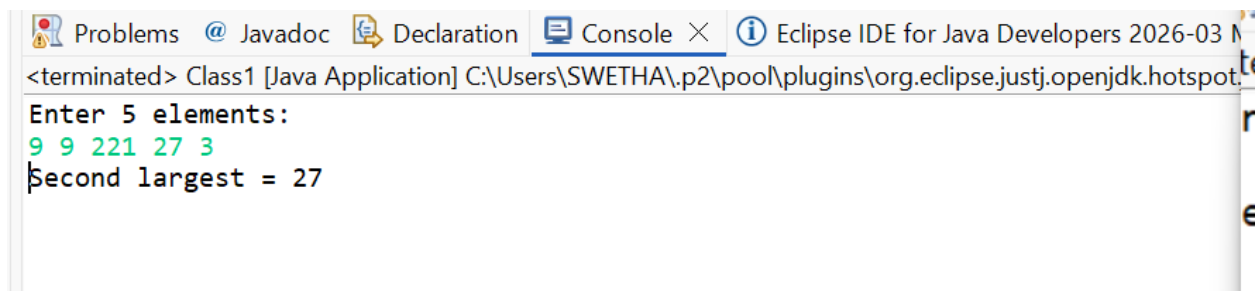


```

for (int i = 0; i < 5; i++) {
    if (arr[i] > largest) {
        second = largest;
        largest = arr[i];
    } else if (arr[i] > second && arr[i] != largest) {
        second = arr[i];
    }
}
if (second == Integer.MIN_VALUE) {
    System.out.println("No second largest element");
} else {
    System.out.println("Second largest = " + second);
}
}
}

```

OUTPUT:



```

Problems @ Javadoc Declaration Console × Eclipse IDE for Java Developers 2026-03 N
<terminated> Class1 [Java Application] C:\Users\SWETHA\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot
Enter 5 elements:
9 9 221 27 3
Second largest = 27

```

8. Program for Matrix Addition (2D Array)

Code:

```

import java.util.Scanner;

public class MatrixAddition {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int[][] a = new int[2][2];
    }
}

```

```
int[][] b = new int[2][2];
int[][] sum = new int[2][2];
System.out.println("Enter elements of matrix A:");
for(int i = 0; i < 2; i++)
    for(int j = 0; j < 2; j++)
        a[i][j] = sc.nextInt();
System.out.println("Enter elements of matrix B:");
for(int i = 0; i < 2; i++)
    for(int j = 0; j < 2; j++)
        b[i][j] = sc.nextInt();
for(int i = 0; i < 2; i++)
    for(int j = 0; j < 2; j++)
        sum[i][j] = a[i][j] + b[i][j];
System.out.println("Sum matrix:");
for(int i = 0; i < 2; i++) {
    for(int j = 0; j < 2; j++)
        System.out.print(sum[i][j] + " ");
    System.out.println();
}
}
```

OUTPUT:

```
<terminated> Class1 [Java Application] C:\Users\SWETHA\.p2\pool\plugins\org.eclipse.justj.openjdk.hotsp  
Enter elements of matrix A:  
2 3 4 5  
Enter elements of matrix B:  
9 9 21 27  
Sum matrix:  
11 12  
25 32
```

POST LAB EXERCISE

1. Why is array indexing usually started from zero instead of one?

Array indexing starts from **zero** because:

- The index represents the **offset from the starting memory address**
- The first element is at offset **0**, so index = 0
- This makes **address calculation faster and simpler** for the system

2. What happens if we try to access an array element outside its declared size?

- ☐ It causes **undefined behavior**
- ☐ The program may:
 - Produce **wrong output**
 - **Crash** or throw an error
 - Access **unintended memory**
- ☐ In some languages (like Java, Python), it throws a **runtime error**
 - Example: `ArrayIndexOutOfBoundsException`

3.How does memory allocation differ for static arrays and dynamic arrays?

Static Array	Dynamic Array
Size fixed at compile time	Size decided at runtime
Memory allocated in stack	Memory allocated in heap
Cannot be resized	Can be resized
Faster access	Slightly slower due to overhead
Example: <code>int a[10];</code>	<code>int* a = malloc(10*sizeof(int));</code>

4.Why is searching faster in arrays compared to linked lists?

- ☐ Arrays support **direct access using index**
- ☐ Binary search can be applied (if sorted)
- ☐ Linked lists require **sequential traversal**
- ☐ Hence, arrays offer **faster searching**

5.What is the difference between contiguous and non-contiguous memory allocation?

Contiguous Memory	Non-Contiguous Memory
Memory blocks are consecutive	Memory blocks are scattered
Faster access	Slower access
Used in arrays	Used in linked lists
Simple address calculation	Needs pointers

Contiguous Memory

Example: Array

Non-Contiguous Memory

Example: Linked List

Result:

Thus the array operations were executed successfully.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	5	
In Lab Exercise	10	
Post Lab Exercise	5	
Viva	10	
Total	30	
Faculty Signature		